Project Report

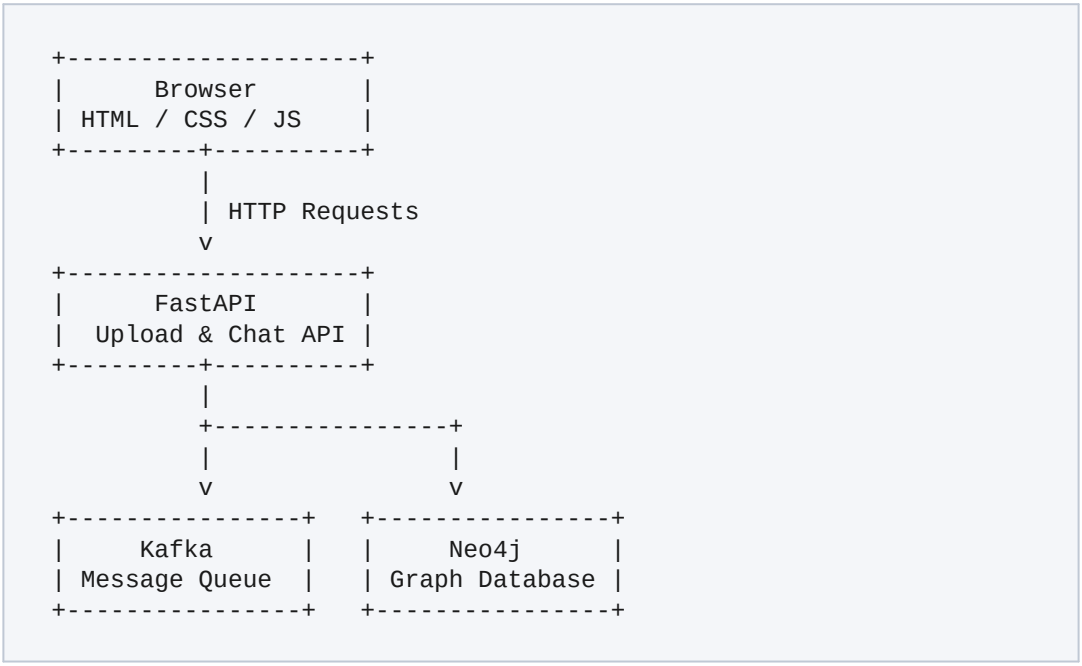
CSV-to-Kafka-to-Neo4j Chatbot Application

Sections 9.1 – 9.2: System Overview, Data & Graph Model

9.1 What We Built

Our team is developing a CSV-to-Kafka-to-Neo4j chatbot application that allows users to upload CSV datasets and query them through a web interface. The frontend was built using HTML, CSS, JavaScript, and Bootstrap, while the backend was developed using FastAPI. The application currently supports CSV upload, dataset preview, and a chat interface through REST APIs. The project is designed to send uploaded data to Kafka for processing and then store the processed graph in Neo4j, where chatbot queries will be answered. At the current stage, the frontend and FastAPI backend are functional, while Kafka and Neo4j integration are being completed as part of the remaining team tasks.

Architecture



High-level data flow: Browser → FastAPI → Kafka / Neo4j

Current Status

Component	Status
Frontend	■ Working
CSV Upload API	■ Working
CSV Preview	■ Working
Chat API	■ Basic endpoint working
Kafka Integration	■ In Progress

Neo4j Integration	■ In Progress
Natural Language Query	■ In Progress

9.2 The Data and the Graph Model

Dataset Used

The application was tested using CSV datasets uploaded through the web interface. During testing, datasets containing approximately **7,000 rows** were used to verify file upload, parsing, and preview functionality.

Example CSV structure:

- Employee_ID
- Name
- Department
- Salary
- City

(The system can also accept other CSV files with different column structures.)

Graph Model

The intended graph model consists of the following node labels and relationship types.

Node Labels

- Dataset
- Row
- Employee (for employee datasets after enrichment)

Relationship Types

- HAS_ROW
- BELONGS_TO
- WORKS_IN (when employee information is detected)

Node Properties

Dataset

dataset_name
upload_time
total_rows

Row

row_id
original CSV column values

Employee

employee_id
name
department
salary
city

Relationship Properties

HAS_ROW

row_number

BELONGS_TO

dataset_name

WORKS_IN

department

Enrichment Strategy

The loader is designed to inspect CSV column names to determine whether the uploaded dataset represents a known entity such as employees.

For example:

- If columns such as Employee_ID, Name, and Department are detected, the loader creates Employee nodes instead of generic Row nodes.
- If the dataset structure is not recognized, the loader falls back to a generic Dataset → Row graph model so that no data is lost.

If automatic detection is incorrect or the dataset does not match any supported schema, the system stores all records as generic Row nodes while preserving the original CSV data.

This approach ensures that every uploaded dataset can still be represented in the graph, even when enrichment is not possible.